

Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Mowbray

Faculty: Information Technology

Module Code: ITECA3-12

Group: 1

Lecturer's Name: Mr. Ndabenhle Ndabana

Student Full Name: Joshua Nehohwa

Student Number: EDUV4948467

Indicate	Yes	No
Plagiarism report attached		

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature jnehohwa	Date 26-May 2026
-----------------------	---------------------

Lecturer's Comments:

--

Marks Awarded:	%
----------------	---

Signature	Date
-----------	------

Eduvos (Pty) Ltd. (formerly Pearson Institute of Higher Education) is registered with the Department of Higher Education and Training as a private higher education institution under the Higher Education Act, 101, of 1997. Registration Certificate number: 2001/HE07/008

ITECA3-12 Deliverable 2: KasiSwap Prototype and Design Evidence

PHP/MySQL C2C marketplace prototype for township and informal-economy users

Live website: <https://kasiswap.free.nf/> | **GitHub submission repo:** <https://github.com/jnehohwa/kasiswap-php-mysql-deliverable-2>

Formal submission note: this report uses the hosted KasiSwap website and the clean PHP/MySQL repository for assessment. It does not rely on localhost, and it does not submit the separate TypeScript/React portfolio prototype.

Introduction

I built KasiSwap as a Customer-to-Customer e-commerce prototype for South African township and informal-economy users. The main idea is simple: buyers and sellers should be able to trade goods online with more trust than they would get from screenshots, off-platform chats, and manual EFT proof. The prototype supports searchable listings, in-app messaging, protected order states, seller verification, reviews, disputes, and an admin website.

The submitted implementation is the clean PHP/MySQL repository, not the separate TypeScript/React portfolio prototype. It uses PHP, MySQL, HTML, CSS, and a small amount of JavaScript. I kept the interface responsive and low-data conscious because the target users are likely to use phones, prepaid data, and slower connections.

Important prototype note: the payment-hold flow is a sandbox simulation for academic demonstration. It models escrow-like order states, but it is not a real regulated escrow or payment service.

Mark Allocation Criteria Cross-Check

I have structured this submission against the exact 50-mark allocation, so each required item is easy to find. The live website and the clean PHP/MySQL GitHub repository are included because the brief does not allow a localhost-only submission.

Criteria	Description	Mark	Submission Evidence
Introduction	Overview of the prototype and technical stack.	2	Introduction states the C2C problem, the live prototype purpose, and the PHP, MySQL, HTML, CSS, and JavaScript stack.
2.1 Prototype	C-2-C platform	6	Buyer and seller flows are documented with marketplace, listing detail, checkout, orders, disputes, messages, and review evidence.
2.1 Prototype	Organization admin website	6	Admin dashboard, user management, verification, listing moderation, disputes, and audit logs are documented as a separate admin area.
2.2 Design Diagrams	Class Responsibility Collaborator (CRC) cards	3	CRC table covers User, SellerProfile, Listing, Order, Payment, Dispute, and Admin responsibilities.

2.2 Design Diagrams	Enhanced Entity Relationship Diagram (EERD)	3	EERD figure shows users, listings, orders, payments, disputes, reviews, messages, verification, audit logs, and relationships.
2.2 Design Diagrams	Context Diagram	3	Context figure shows Buyer, Seller, Admin, KasiSwap platform, MySQL database, and PayFast sandbox simulation.
2.2 Design Diagrams	Data Flow Diagram (DFD)	3	DFD figure shows search, checkout, dispute, moderation, payment state, database, and response flows.
2.2 Design Diagrams	Use Case Diagram	3	Use case figure covers buyer, seller, and admin actions.
2.2 Design Diagrams	Database Design	3	Database design section summarizes the MySQL schema, foreign keys, order states, indexes, and seeded data.
2.3 Coding	Platform Screenshots	2	Main platform screenshots are included as an evidence sheet across mobile, tablet, and desktop, plus key user workflows.
2.3 Coding	Sample PHP Code	3	PHP samples show prepared statements, password verification, session regeneration, RBAC support, and protected order state logic.
2.3 Coding	Sample HTML Code	3	Listing-card markup shows escaped PHP output, semantic article structure, image alt text, and low-data loading.
2.3 Coding	Sample JavaScript Code	3	JavaScript sample shows progressive enhancement for mobile navigation and form behaviour.
2.3 Coding	Sample CSS Code	3	CSS sample shows responsive marketplace grid behaviour from desktop to mobile.
2.3 Coding	Sample MySQL Table Screenshots	2	MySQL/Adminer screenshots show the live database tables and seeded users table; SQL sample shows order/payment structure.
2.4 Conclusion	Summary of the design and development phase.	2	Conclusion summarises how the design, prototype, security, database, and live deployment meet Deliverable 2.
Total		50	All required rubric rows are represented in the document.

Needs and Problem Summary

- Township and informal traders need a safer way to buy and sell online without depending only on screenshots, manual EFT proof, or conversations that happen outside the platform.
- Mobile-first access and data cost pressure mean the pages need to be lightweight, responsive, and usable without a heavy JavaScript application.
- C2C trust needs visible seller verification, in-app messaging, payment-state control, admin dispute handling, and audit logs.

2.1 Prototype

KasiSwap has three main user areas: the public/buyer marketplace, the seller workspace, and the admin website. Together they show the complete marketplace loop from browsing an item to creating an order, confirming sandbox payment, shipping, resolving problems, and reviewing the seller.

- Buyer side: browse listings, filter items, view listing details, create protected orders, simulate sandbox payment, open disputes, send messages, and review completed orders.
- Seller side: manage listings and update shipping only after the platform has confirmed payment into the protected order state.
- Admin side: manage user types, verify sellers, moderate listings, resolve disputes, and inspect audit logs for traceability.
- Database side: MySQL stores users, listings, orders, payments, messages, disputes, reviews, verification requests, and audit logs with relationships and indexes.

C-2-C Platform

The C-2-C platform is shown through the buyer and seller flows: browsing listings, opening listing detail, creating a protected order, confirming sandbox payment, tracking the order, using messages, opening disputes, and leaving reviews. The main website screenshots and demo test scenarios are included as direct evidence.

Organization Admin Website

The admin website is shown through the admin dashboard, user type management, seller verification, listing moderation, dispute handling, and audit logs. The admin screenshots, RBAC checks, and seeded admin account prove that this is a separate organization-facing website, not just a public page.

2.2 Design

Class Responsibility Collaborator (CRC) Cards

Class	Responsibilities	Collaborators
User	Authenticate, hold role, verification level, and profile details	Order, Message, Review, VerificationRequest
SellerProfile	Store seller business name, bio, and sales count	User, Listing
Listing	Describe item, price, location, condition, and visibility	Category, ListingImage, Order
Order	Track buyer/seller transaction and state machine	User, Listing, Payment, Dispute, Review
Payment	Track PayFast sandbox reference and confirmation	Order
Dispute	Record issue, status, and admin outcome	Order, User, AuditLog
Admin	Manage user types, verification, disputes, listings, and audit evidence	User, VerificationRequest, AuditLog

Enhanced Entity Relationship Diagram (EERD)

The EERD connects users, seller profiles, listings, orders, payments, disputes, reviews, messages, verification requests, and audit logs. The central relationship is the order: it links buyer, seller, listing, payment status, dispute handling, and review evidence.

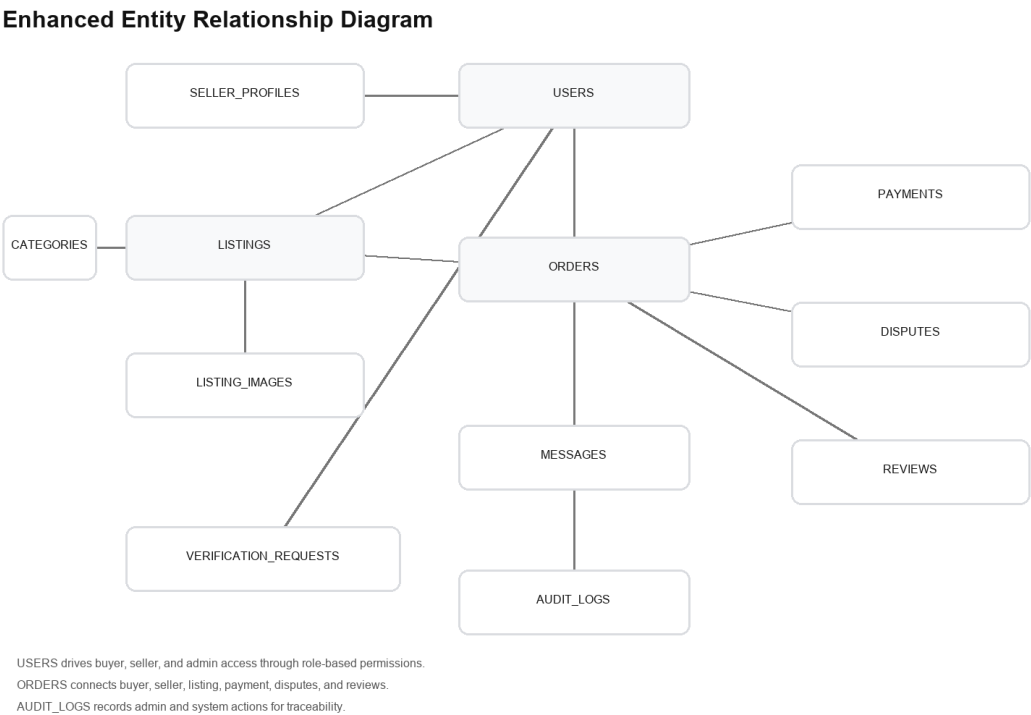


Figure 4: Enhanced Entity Relationship Diagram for the KasiSwap database.

Key relationships and cardinalities:

Relationship	Cardinality	Reason
USERS to SELLER_PROFILES	1 to 0..1	Only seller accounts need a seller profile.
USERS to LISTINGS	1 to many	A seller can create multiple listings.
CATEGORIES to LISTINGS	1 to many	Each listing belongs to a category.
LISTINGS to LISTING_IMAGES	1 to many	A listing can have supporting images.
USERS/LISTINGS to ORDERS	many through orders	Orders connect buyer, seller, and purchased listing.
ORDERS to PAYMENTS	1 to 1	Each protected order has a payment record.
ORDERS to DISPUTES/REVIEWS	1 to many	Orders can produce dispute and review evidence.
USERS to MESSAGES/AUDIT_LOGS	1 to many	Communication and traceability stay linked to users.

Context Diagram

The context diagram shows how buyers, sellers, and admins interact with the platform while the PHP application communicates with MySQL and the sandbox payment-state logic.

Context Diagram

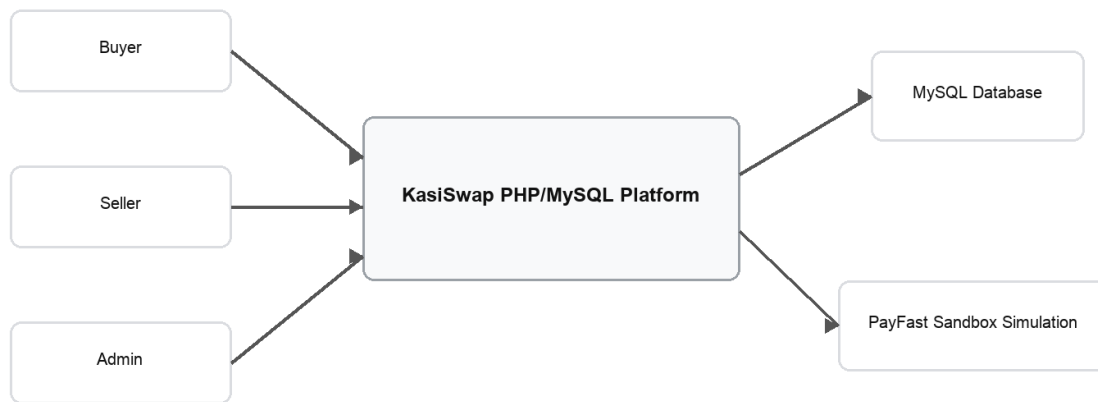


Figure 5: Context diagram showing external actors and platform boundaries.

Data Flow Diagram (DFD)

The DFD shows the practical movement of data: users submit actions through the PHP web app, the app validates requests, prepared statements read or write MySQL data, and users receive updated pages and status messages.

Data Flow Diagram

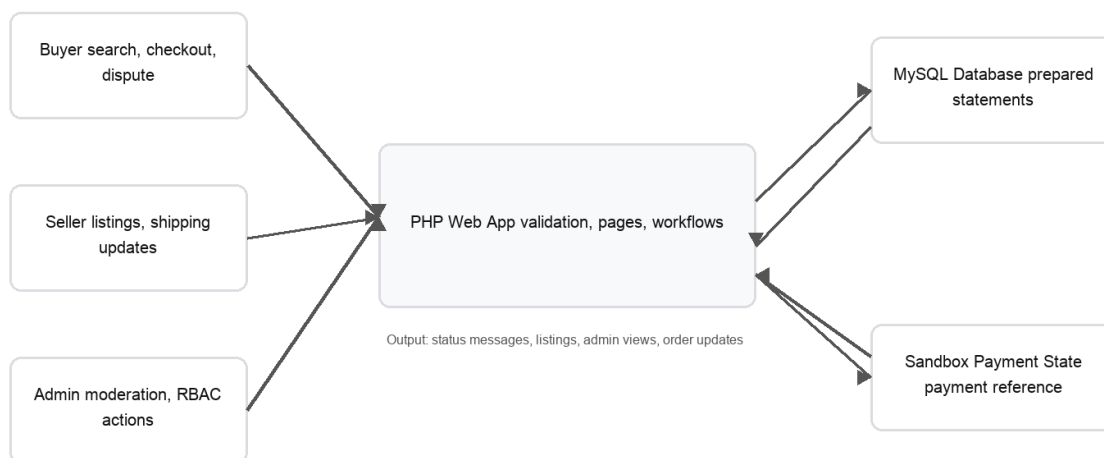


Figure 6: Data flow diagram for marketplace, admin, database, and payment-state interactions.

Use Case Diagram

The use case diagram separates the major buyer, seller, and admin behaviours so the scope of the prototype is clear.

Use Case Diagram

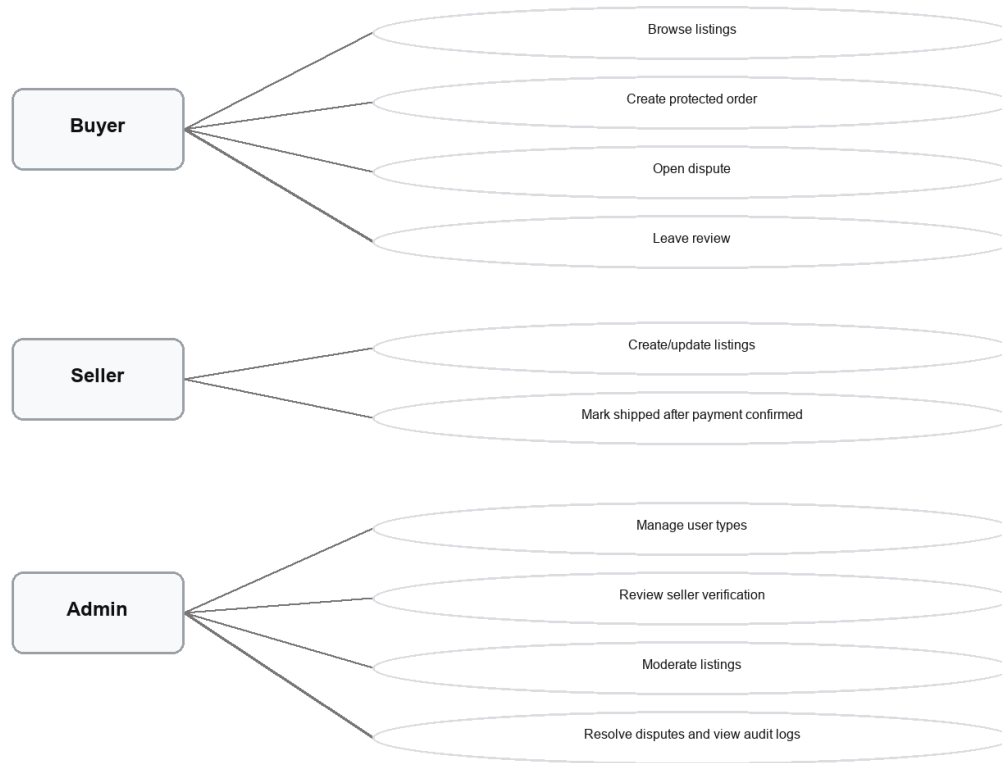


Figure 7: Use case diagram for buyer, seller, and admin workflows.

Database Design

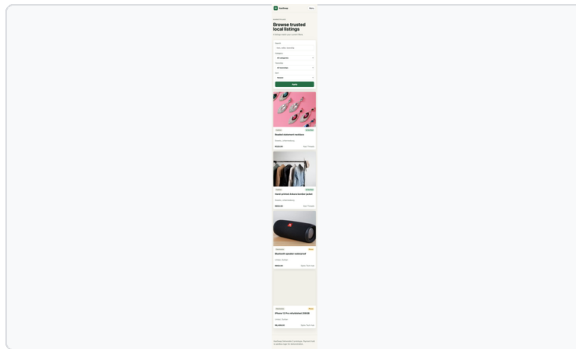
- users: stores accounts, status, role, and verification details for buyers, sellers, and admins.
- seller_profiles: extends seller users with business profile information.
- categories, listings, listing_images: store searchable product data and low-data image evidence.
- orders and payments: track buyer/seller transactions and the sandbox payment-hold state machine.
- disputes and reviews: support trust, accountability, and after-order feedback.
- messages: keeps buyer/seller communication inside the platform.
- verification_requests and audit_logs: support admin review and traceability.

2.3 Coding Evidence

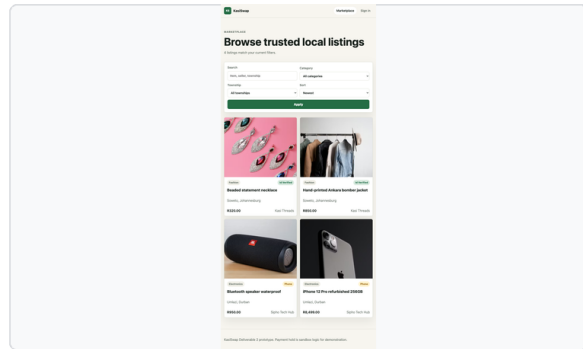
Platform Screenshots and MySQL Table Screenshots

The screenshots below cover the required visual evidence without forcing the assessor to open separate files. I grouped them into evidence sheets so the PDF stays readable while still showing the main site, admin site, and database state.

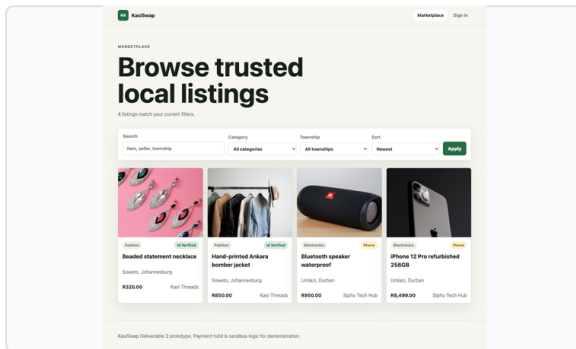
Main C2C Website Screenshot Evidence



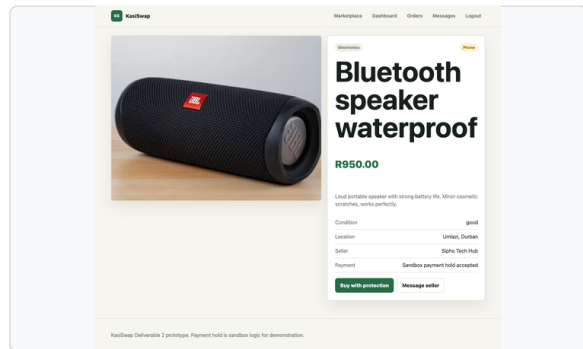
main-marketplace-mobile



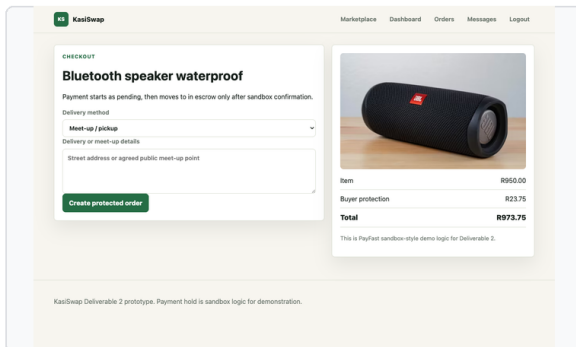
main-marketplace-tablet



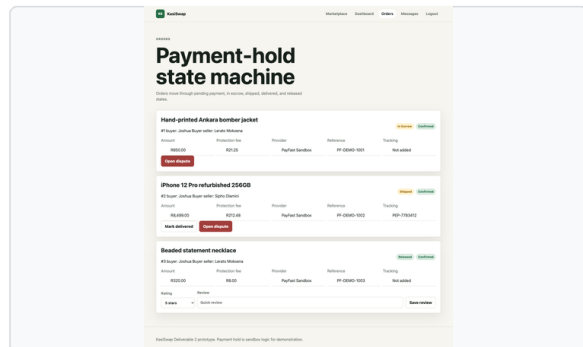
main-marketplace-desktop



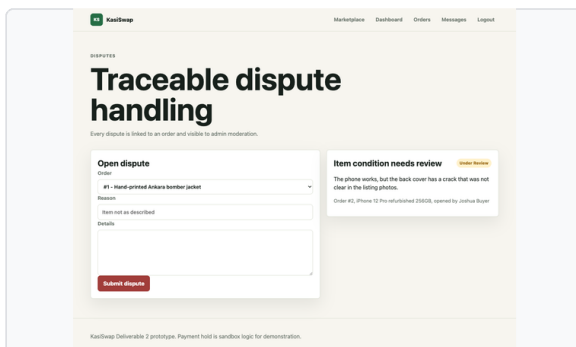
main-listing-detail-desktop



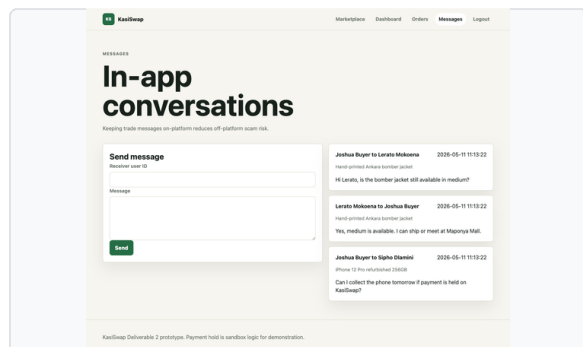
main-checkout-desktop



buyer-orders-desktop



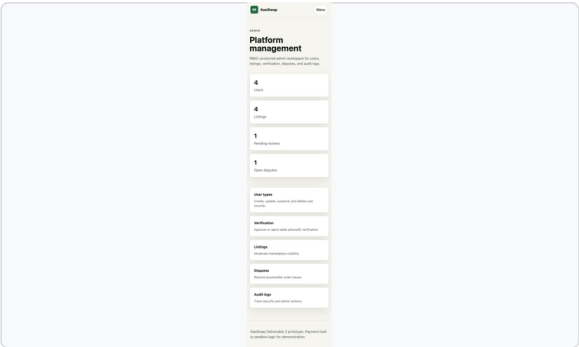
buyer-disputes-desktop



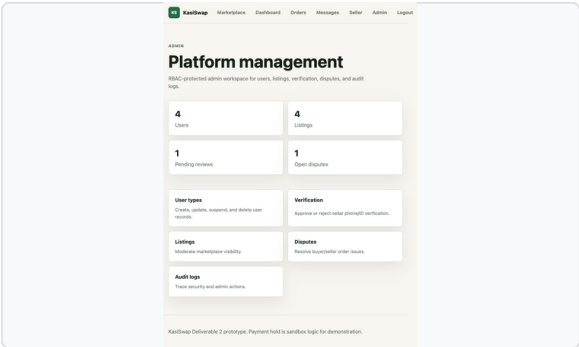
buyer-messages-desktop

Figure 1: Main website evidence: responsive marketplace, listing detail, checkout, buyer orders, disputes, and messages.

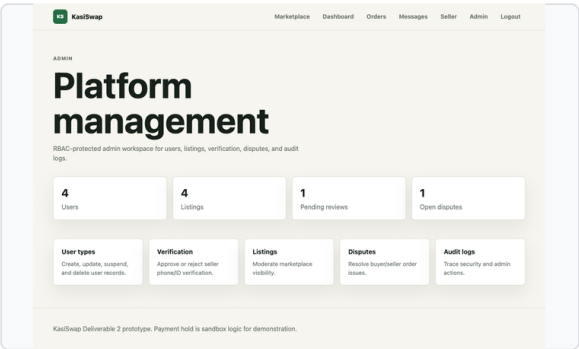
Admin Website Screenshot Evidence



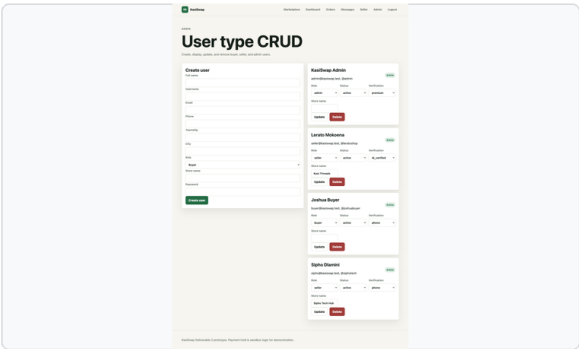
admin-dashboard-mobile



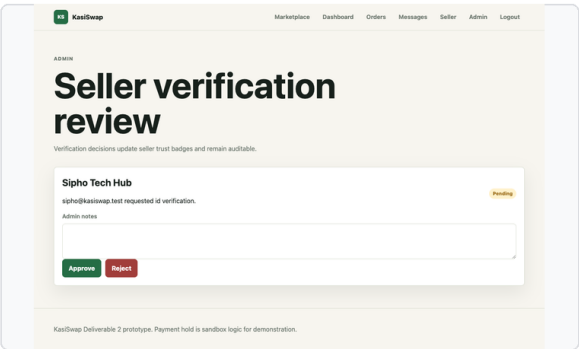
admin-dashboard-tablet



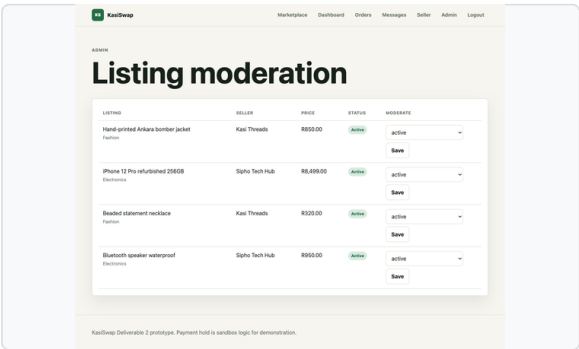
admin-dashboard-desktop



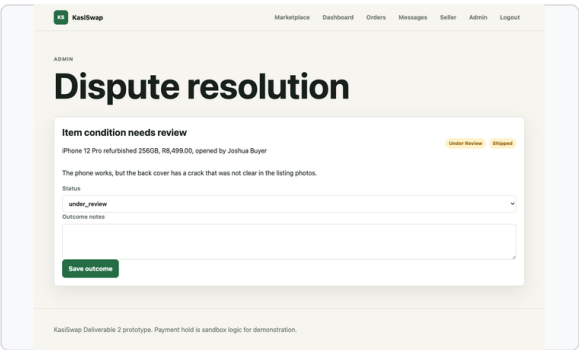
admin-users-desktop



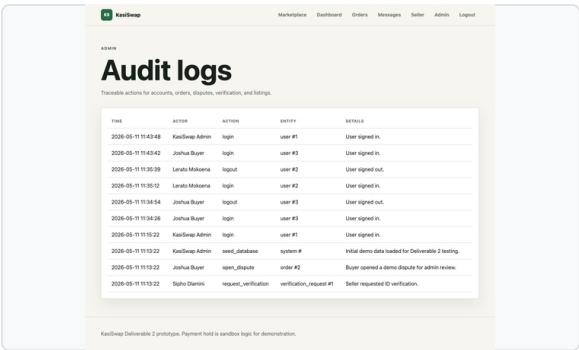
admin-verification-desktop



admin-listings-desktop



admin-disputes-desktop



admin-audit-desktop

Figure 2: Admin website evidence: dashboard, user management, verification, listings, disputes, and audit logs.

MySQL/Adminer Screenshot Evidence

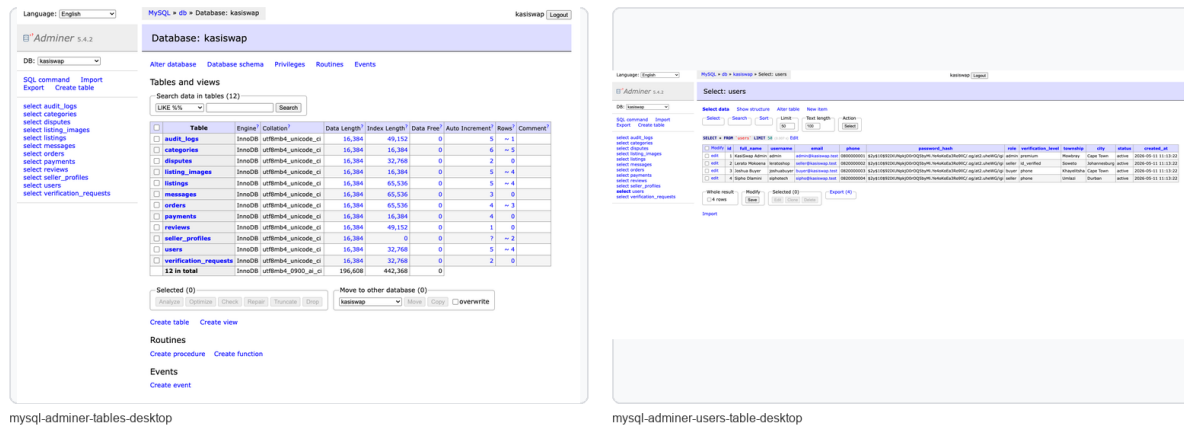


Figure 3: MySQL/Adminer evidence: database tables and seeded users table.

The code samples are intentionally short. Each one shows a marking-relevant part of the implementation without filling the report with entire files.

Sample PHP Code : Secure Login and RBAC

Source file: app/security.php

Why this matters: This shows prepared statements, password verification, session regeneration, and audit logging. Those are core controls for a role-based PHP application.

```
function sign_in(string $email, string $password): bool
{
    $stmt = db()->prepare('SELECT * FROM users WHERE email = ? AND status = "active" LIMIT 1');
    $stmt->execute([$email]);
    $user = $stmt->fetch();

    if (!$user || !password_verify($password, $user['password_hash'])) {
        return false;
    }

    session_regenerate_id(true);
    $_SESSION['user_id'] = (int) $user['id'];
    log_audit('login', 'user', (int) $user['id'], 'User signed in.');
```

Additional PHP Evidence: Fake-Payment Prevention

Source file: app/services.php

Why this matters: This directly supports the project problem: the seller cannot mark an item as shipped unless the order is in escrow and the payment status is confirmed.

```
function seller_mark_shipped(int $orderId, int $sellerId, string $trackingRef): void
{
    $stmt = db()->prepare(
        'SELECT o.*, p.status AS payment_status
        FROM orders o
        LEFT JOIN payments p ON p.order_id = o.id
        WHERE o.id = ? AND o.seller_id = ?'
    );
```

```

$stmt->execute([$orderId, $sellerId]);
$order = $stmt->fetch();

if (!$order || $order['state'] !== 'in_escrow' || $order['payment_status'] !== 'confirmed') {
    throw new RuntimeException('Seller can ship only after platform payment is confirmed in escrow.');
```

```

$stmt = db()->prepare('UPDATE orders SET state = "shipped", tracking_ref = ? WHERE id = ?');
$stmt->execute([trim($trackingRef), $orderId]);
}

```

Sample HTML Code : Listing Card Template

Source file: public/partial/listing-card.php

Why this matters: The listing card escapes output with e() to reduce XSS risk and lazy-loads images to support low-data browsing.

```

<article class="listing-card">
  <a href="/listing.php?id=<?= (int) $listing['id'] ?>">
    " loading="lazy">
  </a>
  <div class="listing-card-body">
    <h3><a href="/listing.php?id=<?= (int) $listing['id'] ?>"><?= e($listing['title'])
?</a></h3>
    <p><?= e($listing['location_township']) ?>, <?= e($listing['location_city']) ?></p>
    <strong><?= money($listing['price']) ?></strong>
  </div>
</article>

```

Sample JavaScript Code : Progressive Enhancement

Source file: public/assets/js/app.js

Why this matters: The JavaScript stays small and supports enhancement instead of replacing the PHP-rendered pages with a heavy client-side application.

```

document.addEventListener("DOMContentLoaded", () => {
  const navToggle = document.querySelector("[data-nav-toggle]");
  const nav = document.querySelector("[data-nav]");

  navToggle?.addEventListener("click", () => {
    nav?.classList.toggle("open");
  });
});

```

Sample CSS Code : Responsive Marketplace Grid

Source file: public/assets/css/styles.css

Why this matters: The responsive grid supports desktop, tablet, and mobile screenshots with one implementation instead of separate layouts.

```

.listing-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 14px;
}

@media (max-width: 980px) {
  .listing-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}

```

```

}

@media (max-width: 720px) {
  .listing-grid {
    grid-template-columns: 1fr;
  }
}

```

Sample MySQL Code and Table Screenshots

Source file: database/01_schema.sql

Why this matters: The schema enforces relationships between buyers, sellers, listings, and orders using foreign keys. The order state enum also makes the protected workflow explicit.

```

CREATE TABLE orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  buyer_id INT NOT NULL,
  seller_id INT NOT NULL,
  listing_id INT NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  state ENUM('pending_payment', 'in_escrow', 'shipped', 'delivered', 'released', 'disputed',
'refunded', 'cancelled') NOT NULL DEFAULT 'pending_payment',
  CONSTRAINT fk_orders_buyer FOREIGN KEY (buyer_id) REFERENCES users(id),
  CONSTRAINT fk_orders_seller FOREIGN KEY (seller_id) REFERENCES users(id),
  CONSTRAINT fk_orders_listing FOREIGN KEY (listing_id) REFERENCES listings(id)
);

```

8. Functional Demo and Testing Evidence

Seeded demo accounts used for testing:

Role	Email	Password
Buyer	buyer@kasiswap.test	password
Seller	seller@kasiswap.test	password
Admin	admin@kasiswap.test	password

The functional testing focuses on whether the prototype works as a complete marketplace and whether the trust controls behave correctly.

1. Buyer logs in, browses listings, opens a listing detail page, creates a protected order, simulates sandbox payment, opens disputes, sends messages, and reviews a completed order.
2. Seller logs in, creates or updates listings, and can mark an order shipped only after payment confirmation moves the order into escrow.
3. Admin logs in, manages user types, approves or rejects seller verification, moderates listings, resolves disputes, and reviews audit logs.
4. Access control is checked by confirming that direct access to /admin/* is blocked for non-admin users.
5. CSRF protection is checked by confirming protected POST forms reject missing tokens.
6. Database evidence is checked in Adminer through populated tables and seeded user data.

9. Live Deployment Evidence

The assignment requires a live hosted website, not a localhost-only prototype. KasiSwap is deployed on InfinityFree with a MySQL database so it can be assessed from the live URL.

- Live website URL: <https://kasiswap.free.nf/>

ITECA_SUMMATIVE_MOWBRAY_EDUV4948467

- GitHub submission repository URL: <https://github.com/jnehohwa/kasiswap-php-mysql-deliverable-2>
- Hosting provider: InfinityFree
- Live database name: if0_41912191_kasiswap
- Database setup: database/01_schema.sql and database/02_seed.sql were imported.
- Submitted repository shape: the root contains only the PHP/MySQL app, database files, docs, screenshots, and Docker setup. It excludes the TypeScript/React portfolio prototype.

10. Security, Trust, and Maintainability

The security work is not treated as an extra feature. It is part of the core design because a C2C marketplace only works if users can trust the platform workflow.

- Prepared statements reduce SQL injection risk across database reads and writes.
- Passwords are checked with password_verify() and stored as hashes instead of plain text.
- session_regenerate_id(true) is used after login to reduce session fixation risk.
- CSRF tokens protect state-changing POST forms.
- Output escaping with e() reduces XSS risk in listing and user-generated content.
- RBAC protects admin routes and separates buyer, seller, and admin abilities.
- Order-state rules prevent sellers from shipping before platform-confirmed sandbox payment.
- Audit logs preserve traceability for login and admin actions.
- config.local.php is kept out of Git so live credentials are not committed.

2.4 Conclusion

KasiSwap demonstrates a working PHP/MySQL C2C marketplace prototype that responds directly to the South African informal-economy scenario. It covers the required buyer, seller, admin, database, diagram, code, testing, and deployment evidence in one submission. The strongest part of the design is that the technical choices connect back to the problem: low-data pages for accessibility, RBAC for role separation, seller verification and disputes for trust, and protected order states to reduce fake-payment handover risk.